



OrbitHam

Estacao Terrena para Rastreamento de Satelites

Global Solution 2026.1 - Graduacao ON em IA

Integrantes

Arthur Prudencio Soares - RM569295

Caroline Coelho Mendes - RM570370

Leandro Paiva - RM572159

Lucas Viana de Lima - RM571835

Matheus Tavares Lima - RM572808

QUERO CONCORRER

1. Introducao

O espaco deixou de ser apenas exploracao cientifica e se tornou um dos maiores ecossistemas de inovacao e transformacao economica do planeta. Satelites monitoram o clima, conectam regioes remotas e apoiam decisoes em escala global. Dentro desse cenario, a comunicacao via satellite por radioamadores depende de operar passagens de satelites de orbita baixa (LEO): janelas curtas em que o satellite fica acima do horizonte da estacao.

Operar uma passagem exige responder, em tempo real, a quatro perguntas: quando o satellite aparece, por quanto tempo, com que qualidade (elevacao) e para onde apontar a antena. Hoje essas informacoes ficam espalhadas em ferramentas diferentes.

O OrbitHam, nossa resposta a Global Solution 2026.1, reúne tudo isso em uma solucao unica e pratica, com tres frentes que compartilham o mesmo motor de propagacao orbital (Skyfield/SGP4): uma aplicacao web, uma camada de analise de dados em Python e a automacao de um rotor de antena com ESP32.

2. Desenvolvimento

2.1 Visao geral da solucao

- Aplicacao web: mapa orbital ao vivo (SGP4 no navegador), predicao de passagens, dashboard com contagem regressiva e apontamento de antena com azimuth, elevacao e correcao Doppler.
- Analise de dados (Python/Jupyter): previsao de decaimento orbital e analise de melhores janelas de operacao, com Pandas, Matplotlib/Seaborn e Machine Learning introdutorio (scikit-learn).
- Automacao (ESP32): um controlador em Python calcula o apontamento e envia por Wi-Fi para um ESP32 que move dois servos, apontando a antena fisicamente para o satellite.

2.2 Arquitetura

O backend segue camadas API -> Service -> Repository -> Model: as rotas nunca acessam o banco diretamente e as entradas/saídas passam por Schemas. A propagacao orbital roda em Python no backend (Skyfield) e no navegador (satellite.js) para o tempo real sem custo de servidor. O mesmo calculo alimenta a analise de dados e o rotor, mantendo as tres frentes coerentes.

OrbitHam - Arquitetura da solucao

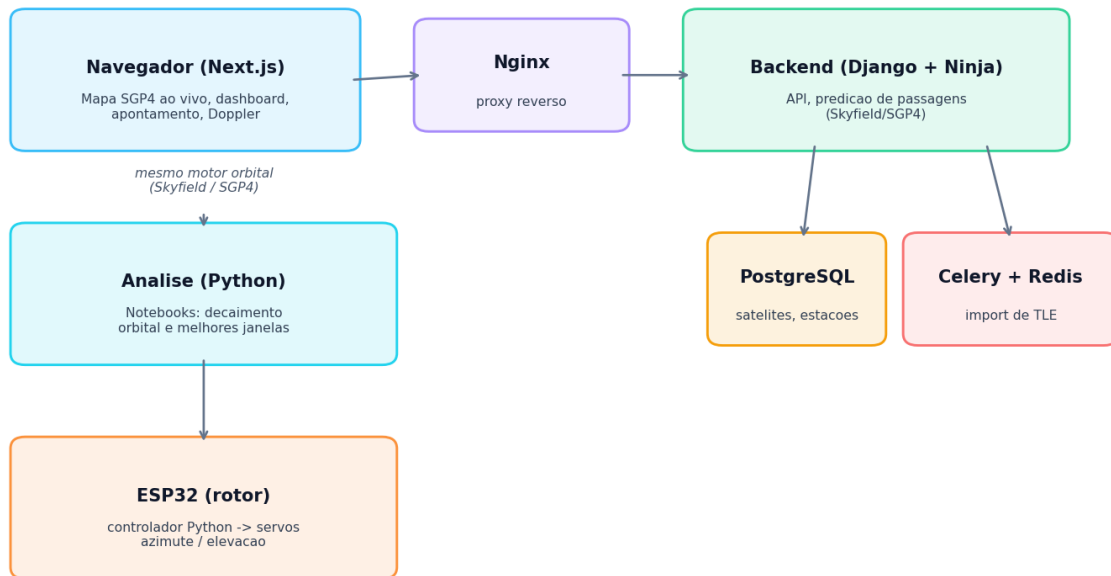


Figura 1. Arquitetura da solucao OrbitHam.

2.3 Codigos principais

Predicao de passagens no backend (Skyfield/SGP4):

```

times, events = satellite.find_events(
    observer, t0, t1, altitude_degrees=10.0)
for ti, event in zip(times, events):
    if event == 0: # subida (rise)
        current = {'rise': ti.utc_datetime()}
    elif event == 1: # pico (culminacao)
        alt, _, _ = (satellite - observer).at(ti).altaz()
        current['max_elevation'] = alt.degrees
    elif event == 2: # descida (set)
        passes.append(current)
  
```

apps/backend/passes/services/pass_prediction_service.py

Apontamento (azimute/elevacao) e distancia ao satellite:

```

alt, az, distance = (satellite - observer).at(t).altaz()
azimute = az.degrees # bussola: para onde apontar
elevacao = alt.degrees # altura acima do horizonte
visivel = elevacao >= 0 # satellite acima do horizonte
  
```

apps/rotor/controller/rotor_controller.py

Regressao do decaimento orbital (analise de dados):

```

coeffs = np.polyfit(daily['day'], daily['altitude_km'], 2)
poly = np.poly1d(coeffs)
# extrapola ate o limiar de reentrada (120 km)
for day in range(6000):
    if poly(day) <= 120.0:
        return day # estimativa de reentrada
  
```

apps/analytics/orbitham_analytics/decay.py

Firmware do ESP32 recebendo o apontamento e movendo os servos:

```

void handlePoint() {
  float az = server.arg("az").toFloat();
  float el = server.arg("el").toFloat();
  azServo.write(map(az, 0, 360, 0, 180));
  elServo.write((int)(el * 2)); // 0..90 -> 0..180
  digitalWrite(LED_PIN, visible ? HIGH : LOW);
}

```

apps/rotor/firmware/rotor_esp32/rotor_esp32.ino

2.4 Decisões do grupo

- Propagação SGP4 no navegador (satellite.js) para o mapa em tempo real, evitando custo de servidor e latência.
- Skyfield/SGP4 em Python no backend e na análise: preciso, testável e executável offline.
- Análise em notebooks Jupyter, reproduzível (dados de TLE embutidos), para evidenciar o passo a passo de ciência de dados.
- Mesmos limites de qualidade de passagem (45 graus e 25 graus) no app e nos notebooks, mantendo produto e análise consistentes.
- Comunicação app/ESP32 por HTTP simples (form-urlencoded), sem bibliotecas extras no firmware.
- Interface inteiramente em português.

3. Resultados Esperados

A solucao entrega um produto utilizavel e uma analise que orienta a operacao. A seguir, os principais resultados.

3.1 Previsao de decaimento orbital

Propagando o TLE da ISS e ajustando uma regressao, o modelo estima uma perda de altitude de cerca de 69 km/ano e uma reentrada por volta de 2027. A comparacao entre satelites mostra que objetos baixos (ISS, ~420 km) decaem rapido, enquanto altos (AO-7, ~1450 km) sao quase estaveis.

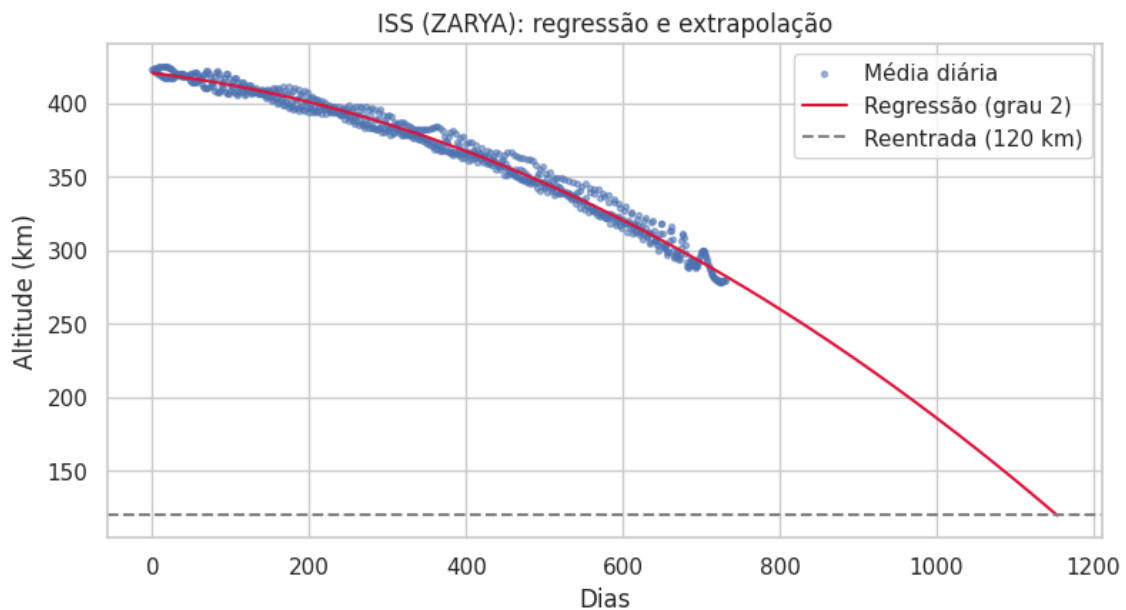


Figura 2. Regressao do decaimento da ISS e limiar de reentrada.

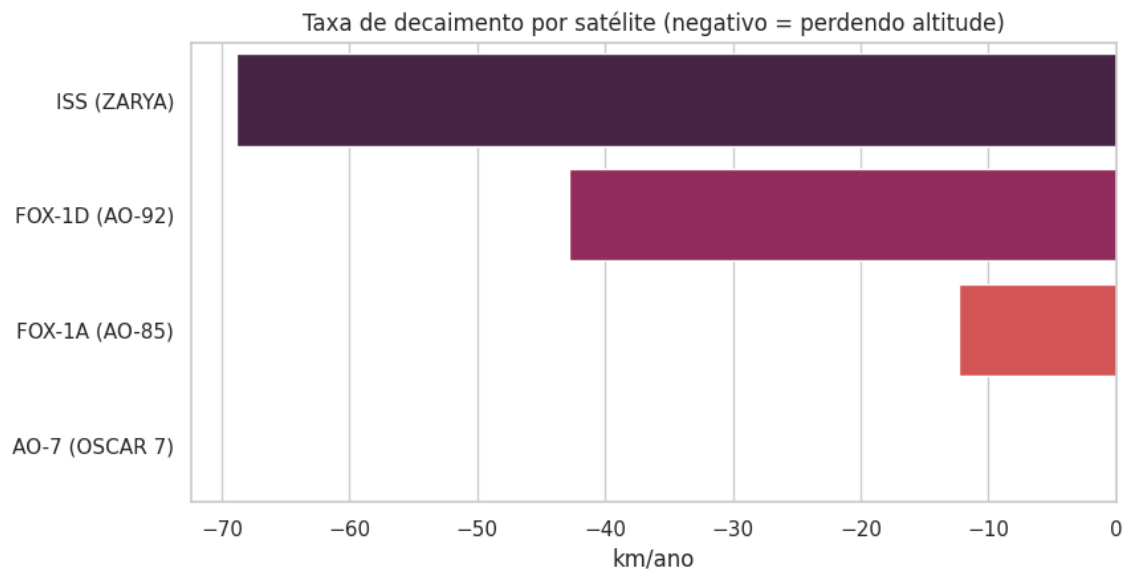


Figura 3. Taxa de decaimento comparada entre satelites.

3.2 Analise de passagens e melhores janelas

Para 182 passagens em 14 dias sobre Sao Paulo, o heatmap revela os melhores horarios de operacao (em torno das 11h e 22h, hora local). Um classificador (arvore de decisao) preve a qualidade da passagem com cerca de 84% de acuracia, e a duracao da passagem e a variavel mais informativa.

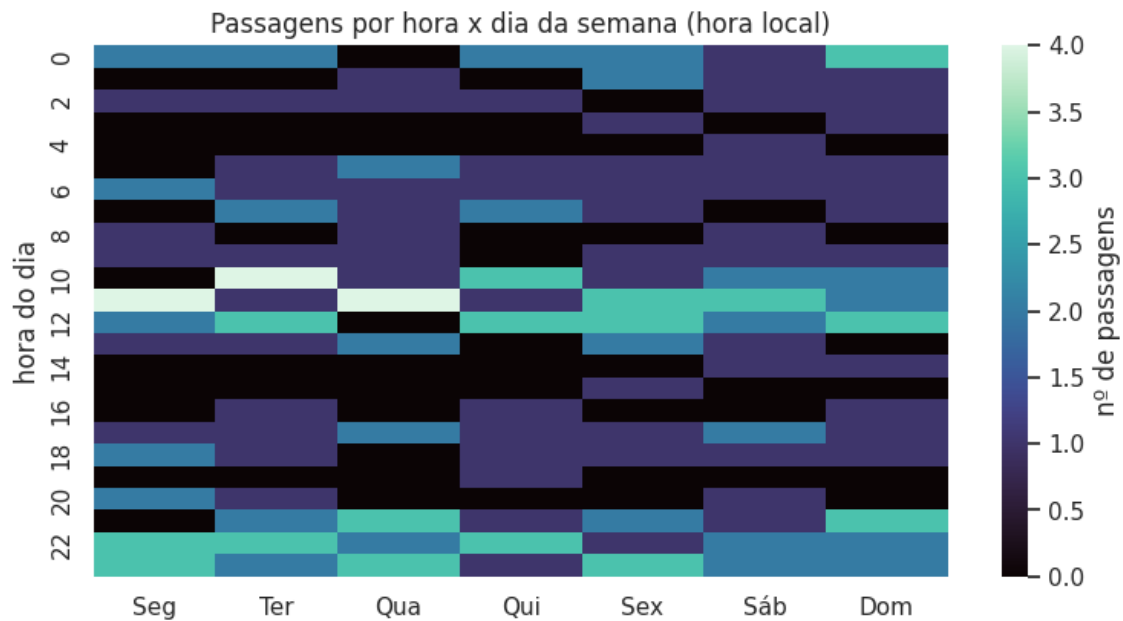


Figura 4. Passagens por hora e dia da semana (melhores janelas).

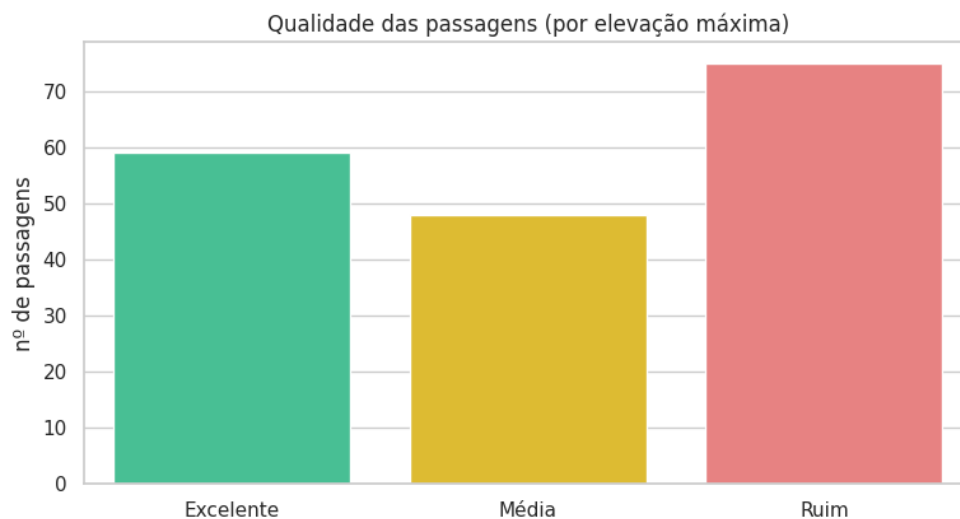


Figura 5. Distribuição de qualidade das passagens.

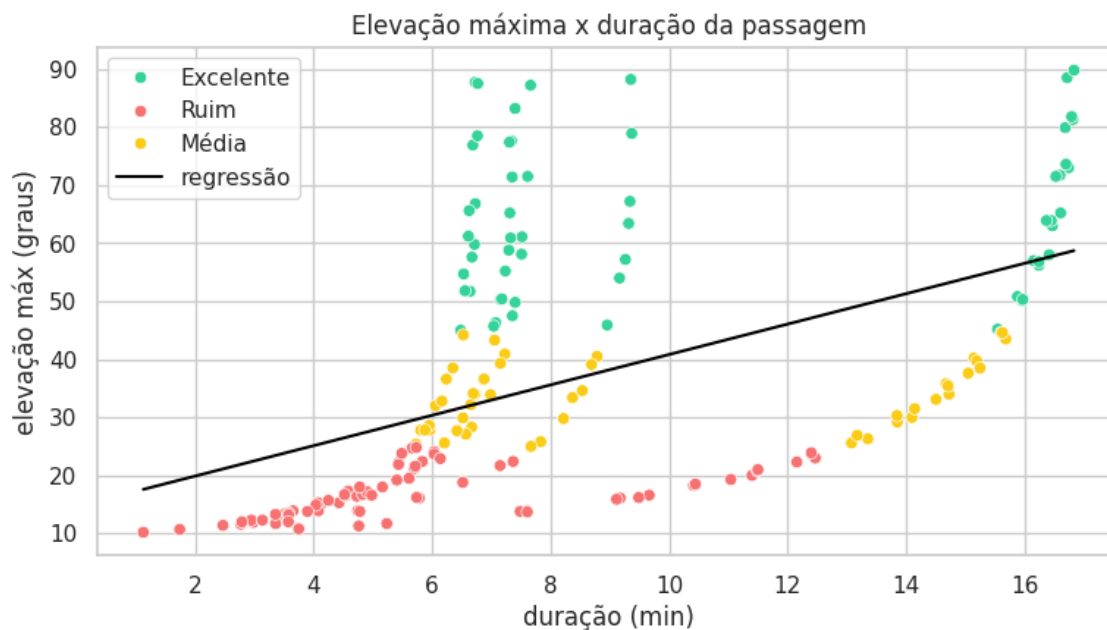


Figura 6. Relação entre duração e elevação máxima (ML).

4. Conclusões

O OrbitHam demonstra, na prática, como a tecnologia espacial melhora processos na Terra: transforma dados orbitais públicos (TLE) em decisões operacionais claras, de quando e para onde apontar a antena até a previsão de decaimento de um satélite.

A solução integra as competências do curso: programação (Python e TypeScript), análise de dados e Machine Learning introdutório, visualização, e automação com ESP32, tudo amarrado pelo mesmo motor orbital. As três frentes se reforçam: o app opera, a análise orienta e o hardware executa o apontamento.

Como evolução, vislumbramos realimentação de posição por sensor IMU no rotor, integração com SDR para recepção real, aplicativo mobile e realimentação da análise com o histórico de TLE já armazenado pelo backend.